

FICHE DE TD 2/3 • MOYEN

Informatique • 1re C D E

Algorithmique : structures de contrôle, fonctions et procédures

Module 2 : Algorithmique et programmation

Cette fiche évalue la maîtrise des structures itératives, des tableaux, de la modularité (fonctions/procédures) et de la portée des variables. Toute trace d'exécution doit être présentée sous forme de tableau clair.

EXERCICE 1 Gestion des notes d'une classe de Première D – Lycée de Bafoussam

Un enseignant souhaite traiter les notes de N élèves ($1 \leq N \leq 50$) stockées dans un tableau `Notes[1..N]` (entiers sur 20). On demande d'écrire les algorithmes correspondants.

1. Déclarer le tableau `Notes` et écrire une procédure `SaisirNotes` permettant de le remplir en contrôlant que chaque note est comprise entre 0 et 20.
2. Écrire une fonction `MoyenneClasse` qui reçoit le tableau et sa taille effective, et renvoie la moyenne de la classe (réel).
3. Écrire une procédure `RechercheNote` qui reçoit le tableau, sa taille et une valeur v à chercher. Elle affiche "Trouvée à l'indice i " pour la première occurrence ou "Absente" sinon.
4. Proposer un algorithme principal utilisant ces modules pour : saisir N , saisir les notes, afficher la moyenne, demander une note à chercher et lancer la recherche.

EXERCICE 2 Modes de passage de paramètres et portée des variables

On considère l'algorithme suivant écrit en pseudo-code :

Variables globales : `X, Y : Entier`

Procédure `Modifier(Valeur A : Entier; Variable B : Entier)`

`A <- A + 10`

`B <- B * 2`

`X <- X + 5`

Afficher "Dans Modifier : A=", `A`, " B=", `B`, " X=", `X`, " Y=", `Y`

Fin Procédure

Algorithme Principal

`X <- 3`

`Y <- 4`

Afficher "Debut Principal : X=", `X`, " Y=", `Y`

`Modifier(X, Y)`

```

Afficher "Après Modifier : X=", X, " Y=", Y
Modifier(Y, X)
Afficher "Fin Principal : X=", X, " Y=", Y
Fin Algorithme

```

1. Identifier le mode de passage de chaque paramètre formel (A et B) de la procédure Modifier. Justifier par les mots-clés utilisés.
2. Compléter le tableau de trace d'exécution ci-dessous (copier le tableau sur votre copie).

Étape	X (Global)	Y (Global)	A (Local)	B (Local/Réf)	Affichage
Début Principal	3	4	-	-	Début Principal :
Appel 1 : Modifier(X, Y)					
Après Appel 1			-	-	
Appel 2 : Modifier(Y, X)					
Après Appel 2			-	-	

3. Expliquer pourquoi la valeur finale de X diffère de celle de Y en lien avec les modes de passage.

EXERCICE 3 Structures de données et algorithme de tri simple – Marché de Ngaoundéré

Un commerçant souhaite informatiser la gestion de son stock de sacs de riz. Chaque sac est caractérisé par : *Marque* (chaîne), *Prix* (entier en FCFA), *Quantité* (entier).

1. Parmi enregistrement, tableau, pile, liste, quelle structure de données convient le mieux pour modéliser un seul sac ? Justifier.
2. Quelle structure choisir pour stocker l'ensemble des N sacs du stock ? Justifier.
3. On déclare le tableau `Stock[1..N]` de ce type enregistrement. Écrire une procédure `TrierParPrixCroissant(Variable T : Tableau; N : Entier)` qui trie le tableau sur place par prix croissant en utilisant l'algorithme du tri par sélection (ou bulle). Préciser la condition d'échange.
4. Écrire une fonction `RechercheDichotomique(T, N, PrixCible)` qui suppose le tableau trié et renvoie l'indice du sac ou 0 si absent. Donner la complexité en temps dans le pire cas.

EXERCICE 4 Calcul numérique et fonction récursive – Physique : Chute libre

On étudie la hauteur $h(t) = h_0 - \frac{1}{2}gt^2$ d'un objet lâché sans vitesse initiale du haut d'un immeuble de 80 m à Douala ($g = 10 \text{ m s}^{-2}$). On souhaite déterminer le temps de chute entier t (en secondes) au sol ($h \leq 0$).

1. Écrire une fonction `Hauteur(t : Entier) : Reel` qui calcule $h(t)$.
2. Écrire une procédure `TempsChute(Variable t : Entier)` qui utilise une boucle `TantQue` pour incrémenter t tant que la hauteur est strictement positive, en appelant la fonction `Hauteur`.
3. Tracez l'exécution de `TempsChute` pour t initialisé à 0. Présentez un tableau avec les colonnes : Itération, t , `Hauteur(t)`, Condition ($H > 0$).
4. Modifier l'algorithme pour qu'il affiche également la vitesse à l'impact $v = g \times t$ (en m/s).

EXERCICE 5 Analyse d'un algorithme complet – Billetterie Camrail

L'algorithme ci-dessous gère la vente de billets pour un train Yaoundé-Ngaoundéré. `Places` est un tableau d'entiers (0 = libre, 1 = occupé) de taille `MAX=100`.

Variables globales : `MAX=100`, `Places[1..MAX] : Entier`, `NbVendus : Entier`

Procédure Initialiser

```
Pour i de 1 à MAX faire Places[i] <- 0 FinPour
NbVendus <- 0
```

Fin Procédure

Fonction TrouverPlaceLibre : Entier

```
i <- 1
TantQue (i <= MAX) ET (Places[i] = 1) faire i <- i + 1 FinTantQue
Si i <= MAX Alors Renvoyer i Sinon Renvoyer 0 FinSi
```

Fin Fonction

Procédure VendreBillet

```
p <- TrouverPlaceLibre()
Si p = 0 Alors Afficher "Complet"
Sinon
    Places[p] <- 1
    NbVendus <- NbVendus + 1
    Afficher "Place ", p, " vendue. Total : ", NbVendus
FinSi
```

Fin Procédure

Algorithme Principal

```
Initialiser()
Repeter
    VendreBillet()
JusquA (NbVendus = MAX)
```

Fin Algorithme

1. Quel est le rôle de la variable globale `NbVendus`? Est-elle modifiée dans `TrouverPlaceLibre`?
2. La fonction `TrouverPlaceLibre` utilise-t-elle des variables locales? Lesquelles? Quelle est leur portée?
3. Simuler l'exécution des 3 premiers appels à `VendreBillet` en montrant l'état du tableau `Places` (indices 1 à 5) et la valeur de `NbVendus` après chaque appel.
4. L'algorithme principal contient une boucle `Répéter...Jusqu'à`. Quelle condition d'arrêt garantit la fin du programme? Que se passe-t-il si on appelle `VendreBillet` une fois de plus après la fin de cette boucle?
5. Proposer une modification de `TrouverPlaceLibre` pour qu'elle renvoie une place aléatoire parmi les libres (indice : utiliser `Aleatoire(1, MAX)` et une boucle). Expliquer pourquoi la version actuelle favorise toujours les places de faible indice.